

SEMINARIUM DYPLOMOWE INŻYNIERSKIE

- Część II

Rozwój portalu wspierającego obsługę rejsów dla biura Armatora
Wydział Oceanografii i Geografii UG

- Bartłomiej Krawisz 193319
- Paweł Pstrągowski 193473
- Stanisław Nieradko 193044

AGENDA

1. Harmonogram prac
2. Stan pracy
3. Przegląd procesu przepisywania frontendu aplikacji
4. Porównanie narzędzi do testów automatycznych
5. Wybrane szczegóły implementacyjne w aplikacji

HARMONOGRAM PRAC

Termin	Zadanie
13 maja 2025	Zakończenie prac nad refaktorem aplikacji, wypuszczenie nowej wersji 2.0.0
czerwiec 2025 - październik 2025	Implementacja nowych funkcjonalności oraz naprawa znajduwanych błędów
21 września 2025	Wprowadzenie testów automatycznych do projektu
październik 2025 - grudzień 2025	Zapoznanie kolejnego zespołu z projektem
październik 2025 - grudzień 2025	Opis prac związanych z projektem w postaci pracy inżynierskiej
listopad 2025 - grudzień 2025	Wdrożenie nowej wersji aplikacji oraz ew. naprawa krytycznych błędów

Równolegle zaplanowane są regularne konsultacje z promotorem oraz klientami w interwałach odpowiednio 2 tygodni oraz miesiąca.

STAN PRACY

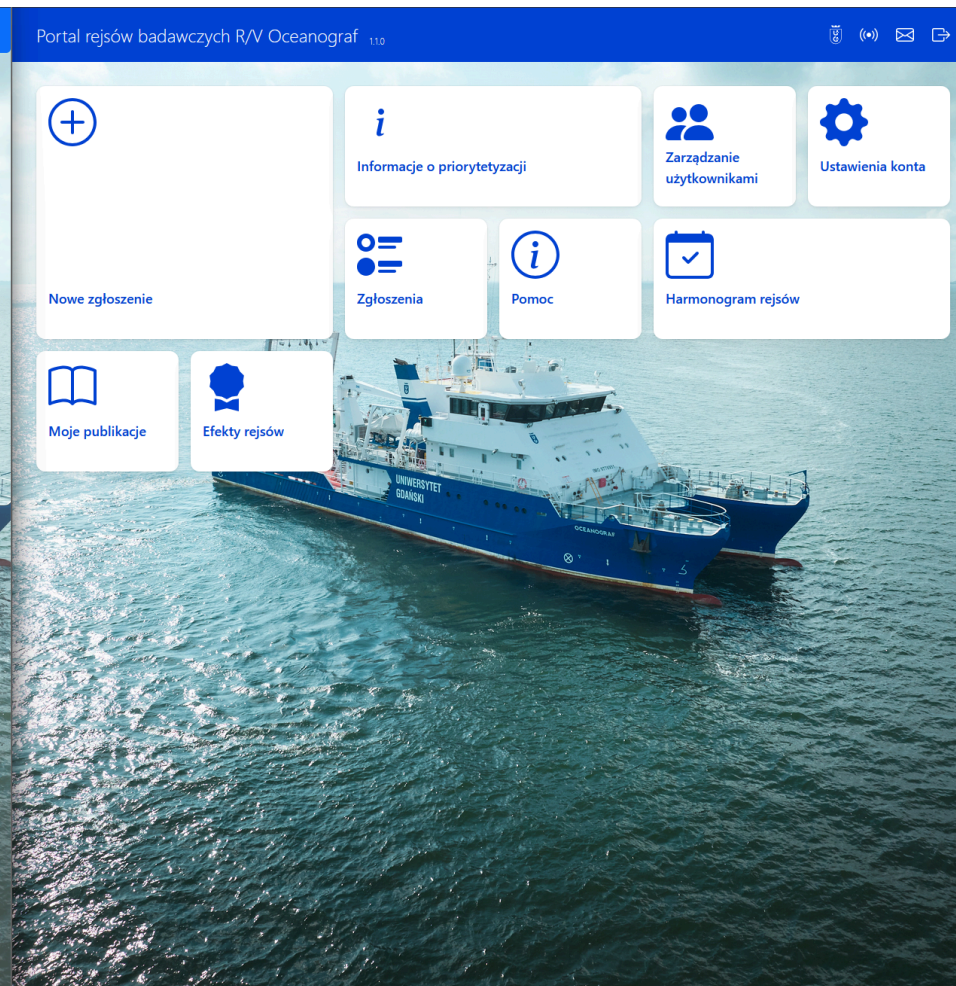
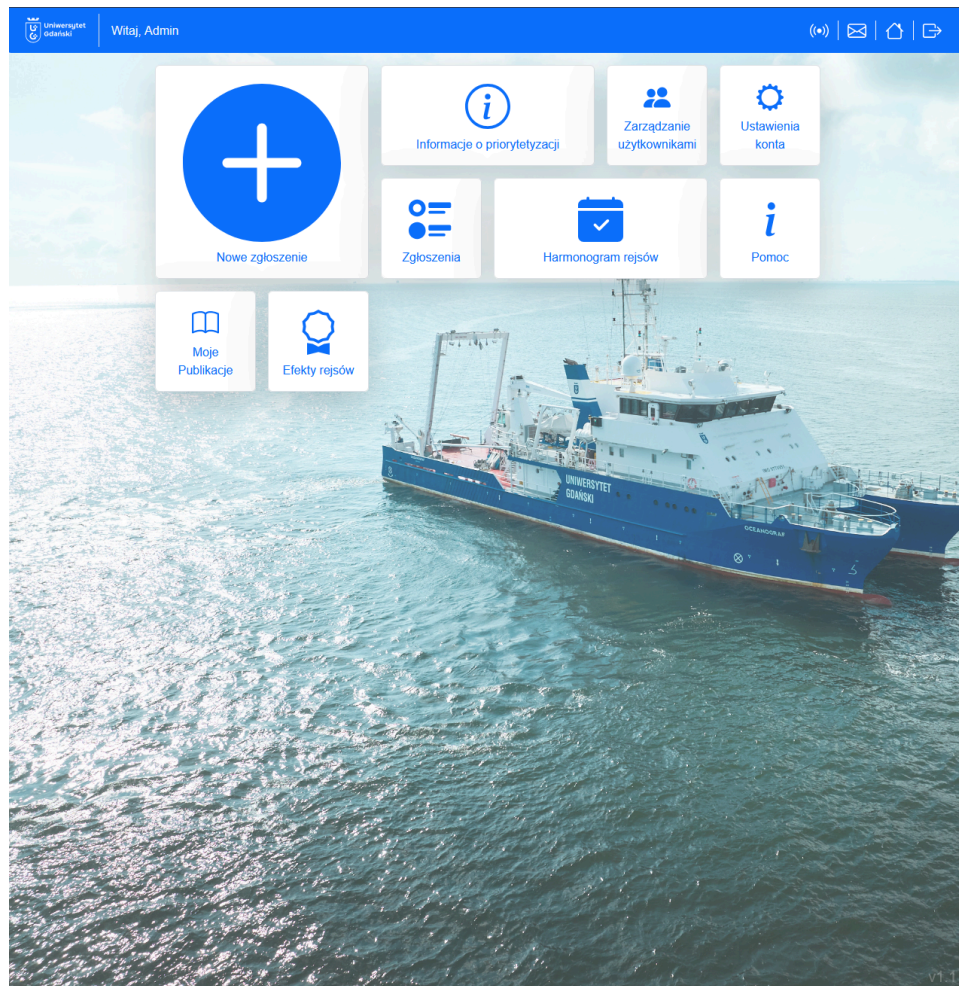
- Większość prac związanych z nowymi funkcjonalnościami dobiegła końca.
- Testy automatyczne zgodnie z planem zostały wdrożone do projektu oraz pipeline'u CI.
- Aktualnie zajmujemy się:
 - Wdrażaniem nowej wersji na środowisko klienta
 - Wprowadzaniem nowego zespołu, który ma przejąć obowiązek utrzymywania aplikacji
 - Opisywaniem naszego projektu w pracy inżynierskiej.

- Nasz zespół zakończył już pracę nad dodawaniem nowych funkcjonalności.
- Testy automatyczne zgodnie z planem zostały wdrożone do projektu oraz pipeline'u CI.
- Nowa wersja aplikacji została wdrożona na środowisko klienta 20 listopada 2025.
- Nowy zespół został wdrożony w projekt, pracują oni aktualnie nad zgłoszonymi problemami i kolejnymi funkcjonalnościami.
- Większość naszej pracy inżynierskiej została już napisana, pozostały tylko końcowe poprawki.

PRZEGLĄD PROCESU PRZEPISYWANIA FRONTENDU APLIKACJI

Porównajmy ponownie UX/UI przed i po

9/52



University of Gdańsk

1. Kierownik2. Czas3. Pozwolenia4. Rejon5. Cel6. Zadania7. Umowy8.2. badawcze9. Publikacje10. SPUB11. Przelazony

Formularz A

1. Kierownik zgłaszającego rejsu

Kierownik rejsu

Admin Adminowy (admin@gmail.com)

Zastępca

Wyszukaj

Rok rejsu

2025

2. Czas trwania zgłaszającego rejsu

Dopuszczalny okres, w którym miałby się odbywać rejs

Wybrano okres: od początku 1. połowy stycznia do końca 2. połowy grudnia.

Liczba planowanych dob rejsowych

0

Optimalny okres, w którym miałby się odbywać rejs

Wybrano okres: od początku 1. połowy stycznia do końca 2. połowy grudnia.

Liczba planowanych godzin rejsowych

0

Uwagi dotyczące terminu

Dodaj uwagi

Statek na potrzeby badań będzie wykorzystywany:

całą dobę

jedynie w ciągu dnia (maks. 8-12 h)

jedynie w nocy (maks. 8-12 h)

8-12 h w ciągu doby rejsowej, ale bez znaczenia o jakiej porze albo z założenia o różnych porach

w inny sposób

Inny sposób użycia

Plan podany inny sposób, ustalony z innymi osobami

3. Dodatkowe pozwolenia do planowanych podczas rejsu badań

Lp.	Treść pozwolenia	Organ wydający pozwolenie
Nie dodano żadnego pozwolenia		

ZapiszWyślij

Portal rejsów badawczych R/V Oceanograf110

Formularz A

1. Kierownik zgłaszającego rejsu

Kierownik rejsu

Admin Adminowy (admin@gmail.com)

Zastępca kierownika rejsu

Wybierz zastępcę kierownika rejsu

Rok

2025

2. Czas trwania zgłaszającego rejsu

Dopuszczalny okres, w którym miałby się odbywać rejs

Wybrano okres: Cały rok

Liczba planowanych dob rejsowych

0

Optimalny okres, w którym miałby się odbywać rejs

Wybrano okres: Cały rok

Liczba planowanych godzin rejsowych

0

Uwagi dotyczące terminu

np. "Rejs w okresie wakacyjnym"

Statek na potrzeby badań będzie wykorzystywany

Wybierz

3. Dodatkowe pozwolenia do planowanych podczas rejsu badań

Dodaj pozwolenie

Lp.	Treść pozwolenia	Organ wydający
Nie dodano żadnego pozwolenia.		

ZapiszWyślij

4. Rejon prowadzenia badań

UWr Uniwersytet Gdański

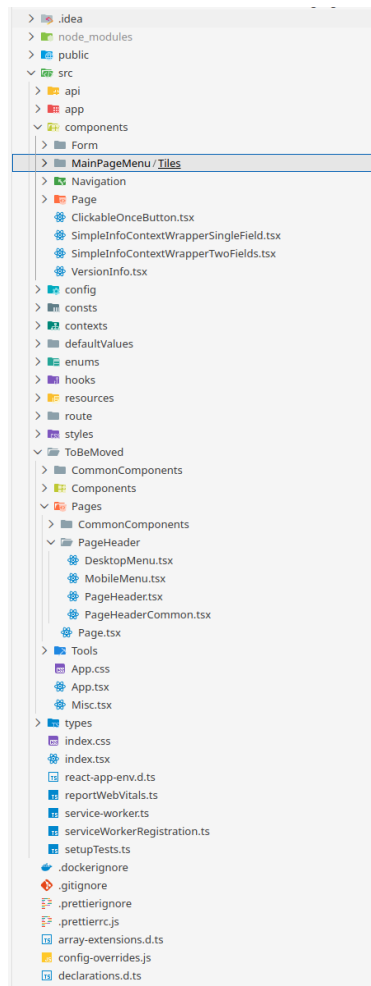
Sortuj	Rok rejsu	Nazwisko kierownika	Filtr wyłączony			
Numer/data	Rok rejsu	Kierownik	Formularze	Punkty	Status	Akcje
7 2025-04-22	2025	Admin Adminowy	Formularz A Formularz B Formularz C	0	Wersja robocza ABC Kontynuuj wypełnianie	Szczegóły
6 2025-04-22	2025	Admin Adminowy	Formularz A Formularz B Formularz C	0	Wersja robocza ABC Kontynuuj wypełnianie	Szczegóły
5 2025-04-22	2025	Admin Adminowy	Formularz A Formularz B Formularz C	0	Wersja robocza ABC Kontynuuj wypełnianie	Szczegóły
4 2025-04-22	2025	Admin Adminowy	Formularz A Formularz B Formularz C	0	Wersja robocza test Kontynuuj wypełnianie	Szczegóły
3 2025-04-22	2025	Admin Adminowy	Formularz A Formularz B Formularz C	0	Wersja robocza Kontynuuj wypełnianie	Szczegóły
2		Admin	Formularz A			

v1.1

Portal rejsów badawczych R/V Oceanograf 110

Wyczyść filtry							
Numer	Data	Rok rejsu	Kierownik	Formularze	Punkty	Status	Akcje
7	2025-04-22	2025	AA Admin Adminowy	Formularz A Formularz B Formularz C	0 pkt.	Wersja robocza ABC Kontynuuj wypełnianie	Szczegóły
6	2025-04-22	2025	AA Admin Adminowy	Formularz A Formularz B Formularz C	0 pkt.	Wersja robocza ABC Kontynuuj wypełnianie	Szczegóły
5	2025-04-22	2025	AA Admin Adminowy	Formularz A Formularz B Formularz C	0 pkt.	Wersja robocza ABC Kontynuuj wypełnianie	Szczegóły
4	2025-04-22	2025	AA Admin Adminowy	Formularz A Formularz B Formularz C	0 pkt.	Wersja robocza test Kontynuuj wypełnianie	Szczegóły
3	2025-04-22	2025	AA Admin Adminowy	Formularz A Formularz B Formularz C	0 pkt.	Wersja robocza Kontynuuj wypełnianie	Szczegóły
2	2025-04-10	2025	AA Admin Adminowy	Formularz A Formularz B	50 pkt.	Zaakceptowane przez przełożonego	Szczegóły

- Poprzednia wersja była mało wydajna, nieczytelna oraz miała wiele błędów UX/UI.
- Użytkownicy doświadczali losowych problemów takich jak wylogowania przez błędną rotację tokenów, utratę stanu aplikacji w trakcie nawigacji czy brak poprawnego wyświetlania błędów.
- Ograniczone filtrowanie/sortowanie oraz chaotyczny UX utrudniały codzienną pracę.
- Kod był niespójny, trudny do utrzymania i (co najgorsze) stylowany jednym plikiem SCSS nadpisującym Bootstrap `(-.-||)`.

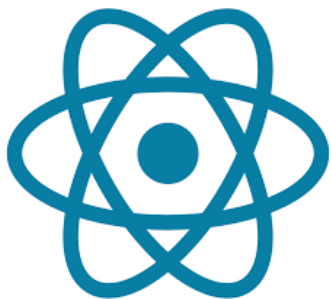


```

src > ToBeMoved > Components > Pages > FormPage > Inputs > PublicationsTable > PublicationsTable.tsx > ...
1 import { CategoryPicker, InformationColumn, MinisterialPointField, YearField } from './PublicationsTableFields';
2 import { ButtonMenuHistory, OrdinalNumber, RemoveReduction } from '@app/pages/FormPage/Inputs/Complex';
3 import { FieldProps } from '@app/pages/FormPage/Inputs/Complex';
4 import { useContext } from 'react';
5 import { FieldLabelWrapper } from '@app/pages/FormPage/Inputs/FieldLabelWrapper';
6 import { FieldValues } from '@app/pages/FormPage/Inputs/FieldLabelWrapper';
7 import { FormContext } from '@app/pages/FormPage/Inputs/FieldLabelWrapper';
8 import { FieldContextWrapper } from '@app/pages/FormPage/Inputs/FieldLabelWrapper';
9 import { FieldWrapper } from '@app/pages/FormPage/Inputs/FieldLabelWrapper';
10
11 export const notEmptyArray = <T extends object>(value: FieldValues) => {
12   if (value.some(row => !row)) {
13     return Object.values(row).some(field => {
14       if (field === null || field === undefined) {
15         return true;
16       }
17     });
18   }
19   return 'Wypełnij wszystkie pola';
20 }
21
22 export type Publication = {
23   category: string;
24   doi: string;
25   authors: string;
26   title: string;
27   magazine: string;
28   year: string;
29   ministerialPoint: string;
30 }
31
32 const publicationDefaultValues = [
33   {
34     category: 'subject',
35     doi: '',
36     authors: '',
37     title: '',
38     magazine: '',
39     year: '0',
40     ministerialPoint: '0',
41   },
42   {
43     category: 'postscript',
44     doi: '',
45     authors: '',
46     title: '',
47     magazine: '',
48     year: '0',
49     ministerialPoint: '0',
50   },
51 ];
52
53 export const publicationCategories = [
54   'subject',
55   'postscript',
56 ];
57
58 export const publicationCategoriesPL = [
59   'temat',
60   'dopisek',
61 ];
62
63 export const publicationOptions = publicationCategoriesPL.map((taskLabel, index) => {
64   ({ label: taskLabel, value: publicationDefaultValues[index] });
65 });
66
67 const PublicationFormContext = () => {
68   [ () => <div>{Number} {label} {Publikacja} </div>,
69     CategoryPicker,
70     InformationColumn,
71     YearField,
72     MinisterialPointField,
73     RemoveReduction,
74   ];
75 }
76
77 type PublicationsTableProps = FieldProps & {
78   historicalPublications?: Publication[];
79 }
80
81 const PublicationFormLabel = (row: Publication) => {
82   DOI: View.doiIn
83   Autorzy: View.authorsIn
84   Tytuł: View.titleIn
85   Czasopismo: View.magazineIn
86   Rok wydania: View.yearIn
87   Punkty: View.ministerialPoints;
88 }
89
90 export const PublicationsTable = (props: PublicationsTableProps) => {
91   const formContext = useContext(FormContext);
92   const filteredHistoricalPublications = (category: string) => {
93     props.historicalPublications?.filter(row => row.category === category)
94   };
95   const filteredPublications = (category: string) => {
96     filteredHistoricalPublications(category)
97   };
98   const selectOptions = publicationCategories.map((publicationCategory, index) => {
99     ({ label: publicationCategoriesPL[index],
100       options: filteredHistoricalPublications(publicationCategory),
101     });
102   });
103
104   const columns = [5, 15, 15, 15, 15, 5];
105   const cellLabels = ['Id.', 'Kategoria', 'Informacja', 'Rok wydania', 'Punkty ministerialne', ''];
106   const cellTitles = ['Publikacja', ''];
107   const bottomMenu = <div>{buttonMenuHistory} {buttonMenuOptions} {buttonMenuOptions} {buttonMenuOptions} </div>;
108   const emptyText = 'Nie dodano żadnej publikacji';
109   const { header } = FieldLabelWrapper({ title: 'Publikacja', columns: columns, cellTitles: cellTitles, PublicationTableContent,
110     bottomMenu, emptyText, formContext, getValues(props.fieldName) });
111
112   const fieldProps = {
113     ...props,
114     defaultValues: [],
115     ...fieldProps,
116   };
117 }

```

- Bardzo dużo powtórzeń kodu.
- Nadmierne wykorzystanie patternów i wzorców projektowych + React.
- Brak spójności w nazewnictwie i strukturze plików.
- Wiele komponentów w jednym pliku.
- Zatrważająco dużo `any` w TypeScript.



React



TypeScript



Vite



Tailwind



Storybook



Motion



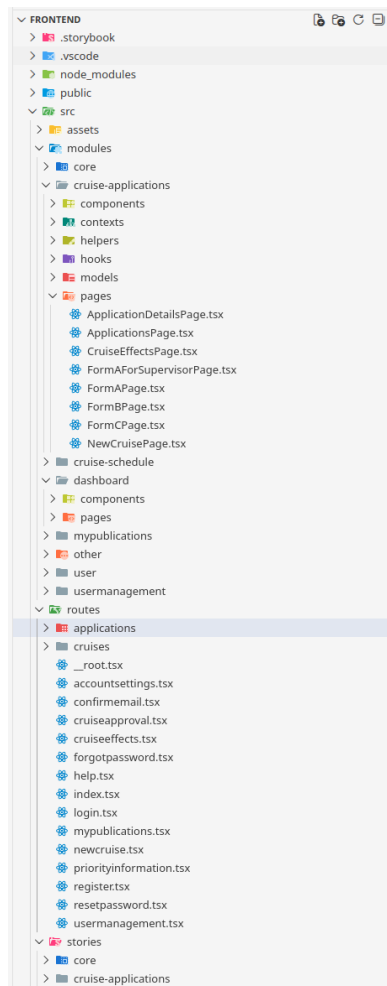
TanStack
Router



Zod

- React + TypeScript jako baza dzięki doświadczeniu zespołu i popularności ekosystemu.
- Vite zamiast create-react-app/react-app-rewired: szybszy build/dev, prostsza konfiguracja.
- Tailwind CSS dla izolowanych, komponentowych stylów; rezygnacja z ciężkiego SCSS nadpisującego Bootstrap.
- Storybook do podglądu i testów komponentów UI, wsparcie dla wdrożenia kolejnych zespołów.
- Motion - animacje i przejścia dla lepszego UX.

- TanStack Router: lepsza integracja z danymi, typowane ścieżki, wbudowane stany ładowania.
- TanStack Query zamiast czystego axios: cache odpowiedzi, retry idempotentnych żądań, praca offline.
- TanStack Forms: reaktywne formularze, walidacja na bazie zod, ochrona przed utratą danych.
- TanStack Table: generyczne filtrowanie/sortowanie, widok list na mobile, wyszukiwarka w tabelach.
- Własna biblioteka komponentów zamiast zewnętrznych UI kits dla pełnej spójności stylu.



src > modules > cruise-applications > components > application-details > ApplicationDetails.tsx > ApplicationDetails > acceptApplication

```
1 import { useQueryClient } from '@tanstack/react-query';
2 import toast from 'react-hot-toast';
3
4 import { ApplicationDetailsProvider } from '@cruise-applications/contexts/ApplicationDetailsContext';
5 import {
6   useAcceptApplicationMutation,
7   useRejectApplicationMutation,
8 } from '@cruise-applications/hooks/CruiseApplicationsApiHooks';
9 import { CruiseApplicationDto } from '@cruise-applications/models/CruiseApplicationDto';
10 import { EvaluationDto } from '@cruise-applications/models/EvaluationDto';
11
12 import { ApplicationDetailsActionsSection } from './ApplicationDetailsActionsSection';
13 import { ApplicationDetailsContractsSection } from './ApplicationDetailsContractsSection';
14 import { ApplicationDetailsEffectPointsSection } from './ApplicationDetailsEffectPointsSection';
15 import { ApplicationDetailsInformationSection } from './ApplicationDetailsInformationSection';
16 import { ApplicationDetailsMembersSection } from './ApplicationDetailsMembersSection';
17 import { ApplicationDetailsPublicationsSection } from './ApplicationDetailsPublicationsSection';
18 import { ApplicationDetailsResearchTasksSection } from './ApplicationDetailsResearchTasksSection';
19 import { ApplicationDetailsSPUBTasksSection } from './ApplicationDetailsSPUBTasksSection';
20
21 type Props = {
22   application: CruiseApplicationDto;
23   evaluation: EvaluationDto;
24 };
25 export function ApplicationDetails({ application, evaluation }: Props) {
26   const queryClient = useQueryClient();
27
28   const acceptMutation = useAcceptApplicationMutation();
29   const rejectMutation = useRejectApplicationMutation();
30
31   function acceptApplication() {
32     acceptMutation.mutate(application.id, {
33       onSuccess: () => {
34         queryClient.invalidateQueries({ queryKey: ['cruiseApplications', application.id] });
35         toast.success('Formularz został zaakceptowany');
36       },
37       onError: (err) => {
38         console.error(err);
39         toast.error('Nie udało się zaakceptować formularza');
40       },
41     });
42   }
43
44   function rejectApplication() {
45     rejectMutation.mutate(application.id, {
46       onSuccess: () => {
47         queryClient.invalidateQueries({ queryKey: ['cruiseApplications', application.id] });
48         toast.success('Formularz został odrzucony');
49       },
50       onError: (err) => {
51         console.error(err);
52         toast.error('Nie udało się odrzucić formularza');
53       },
54     });
55   }
56
57   return (
58     <ApplicationDetailsProvider value={{ application, evaluation }}>
59     <ApplicationDetailsInformationSection />
60     <ApplicationDetailsResearchTasksSection />
61     <ApplicationDetailsEffectPointsSection />
62     <ApplicationDetailsContractsSection />
63     <ApplicationDetailsMembersSection />
64     <ApplicationDetailsPublicationsSection />
65     <ApplicationDetailsSPUBTasksSection />
66     <ApplicationDetailsActionsSection onAccept={acceptApplication} onReject={rejectApplication} />
67     </ApplicationDetailsProvider>
68   );
69 }
70
```



```
src > modules > core > components > AppAccordion.tsx > AppAccordion
1  import ChevronDownIcon from 'bootstrap-icons/icons/chevron-down.svg?react';
2  import ChevronUpIcon from 'bootstrap-icons/icons/chevron-up.svg?react';
3  import { AnimatePresence, motion } from 'motion/react';
4  import React from 'react';
5
6  type Props = {
7    title: string;
8    children: React.ReactNode;
9    expandedByDefault?: true | undefined;
10 };
11
12 export function AppAccordion({ title, children, expandedByDefault = undefined }: Props) {
13   const [expanded, setExpanded] = React.useState<boolean>(!expandedByDefault);
14
15   return (
16     <>
17       <h2 className="w-full">
18         <button
19           type="button"
20           className="w-full flex justify-between items-center cursor-pointer px-4 py-4 bg-black/2 rounded-xl"
21           onClick={() => setExpanded(!expanded)}
22         >
23           <span className="font-semibold text-lg">{title}</span>
24           <span>{expanded ? <ChevronUpIcon className="w-6 h-6" /> : <ChevronDownIcon className="w-6 h-6" />}</span>
25         </button>
26       </h2>
27       <AnimatePresence initial={false}>
28         {expanded && [
29           <motion.div
30             initial={{ opacity: 0, height: 0 }}
31             animate={{ opacity: 1, height: 'auto' }}
32             exit={{ opacity: 0, height: 0 }}
33             transition={{ ease: 'easeOut' }}
34             className="px-4"
35           >
36             {children}
37           </motion.div>
38         ]}
39       </AnimatePresence>
40     </>
41   );
42 }
43
```

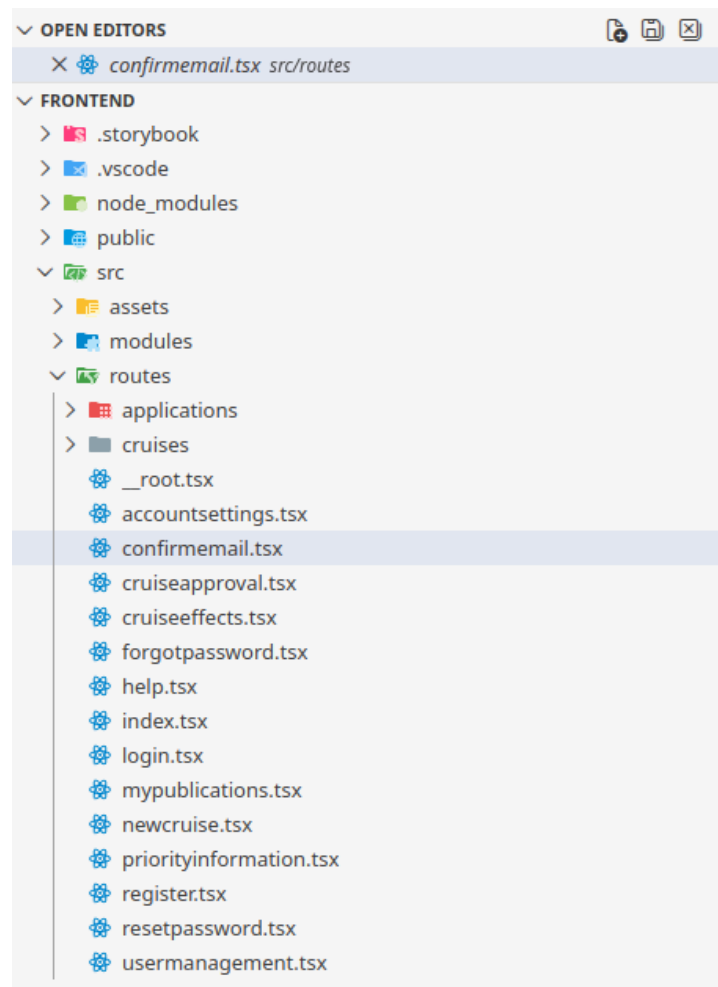
```
type SupervisorFormQueryProps = {
  cruiseId: string;
  supervisorCode: string;
};

export function useFormAForSupervisorInitValuesQuery({ cruiseId, supervisorCode }: SupervisorFormQueryProps) {
  return useSuspenseQuery({
    queryKey: ['formAForSupervisorInitValues', cruiseId, supervisorCode],
    queryFn: async () => {
      return client.get(
        `/forms/InitValuesForSupervisor/A?cruiseApplicationId=${cruiseId}&supervisorCode=${supervisorCode}`
      );
    },
    select: (res) => res.data as FormAInitValuesDto,
  });
}

export function useFormAForSupervisorQuery({ cruiseId, supervisorCode }: SupervisorFormQueryProps) {
  return useSuspenseQuery({
    queryKey: ['formAForSupervisor', cruiseId, supervisorCode],
    queryFn: async () => {
      return client.get(`/api/CruiseApplications/${cruiseId}/formAForSupervisor?supervisorCode=${supervisorCode}`);
    },
    select: (res) => {
      const dto = res.data as FormADto;
      dto.note ??= '';
      return dto;
    },
  });
}

type SaveFormAProps = {
  form: FormADto;
  draft: boolean;
};

export function useSaveFormAMutation() {
  return useMutation({
    mutationFn: async ({ form, draft }: SaveFormAProps) => {
      return client.post(`/api/CruiseApplications?isDraft=${draft}`, form);
    },
  });
}
```



```
src > routes > confirmemail.tsx > ...
1  import { createFileRoute } from '@tanstack/react-router';
2  import { z } from 'zod';
3
4  import { allowOnly } from '@core/lib/guards';
5  import { ConfirmEmailPage } from '@user/pages/ConfirmEmailPage';
6
7  export const Route = createFileRoute('/confirmemail')({
8    component: ConfirmEmailPage,
9    beforeLoad: allowOnly.unauthenticated(),
10   validateSearch: z.object({
11     userId: z.string().uuid().optional(),
12     code: z.string().optional(),
13   }),
14 });
15
```


- Odświeżanie tokenu poza React Query (brak dostępu do kontekstu).
- Walidacja z dwiema konfiguracjami zod w locie okazała się kłopotliwa; błąd naprawiono przy przekazaniu projektu.
- Kalkulacja pozycji dropdownów dla dużych ekranów miała błąd matematyczny (naprawa po 3 dniach debugowania).
- Druk PDF (react-to-print): konieczność duplikacji uproszczonych komponentów formularzy wydłużyła prace.

- Feature parity przy zachowaniu znanego stylu, ale z lepszym UX/UI i stabilnością.
- Wyższa wydajność potwierdzona Lighthouse: lepsze LCP (Largest Contentful Paint) oraz TBT (Total Blocking Time)
- Uporządkowany kod (komponentowe style, wyraźna struktura katalogów) ułatwia utrzymanie.
- Aplikacja dostępna u klienta od 20.11.2025; nowy zespół wdrożony i rozwija kolejne funkcje.

PORÓWNIANIE NARZĘDZI DO TESTÓW AUTOMATYCZNYCH

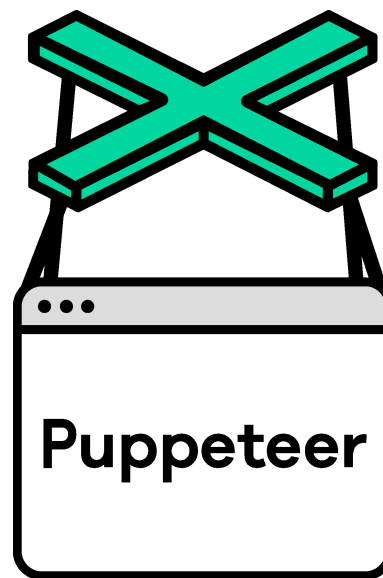
Najpopularniejsze rozwiązania stosowane w branży:



Selenium



Cypress



Puppeteer



Playwright

Selenium

Zalety:

- Szerokie wsparcie dla wielu przeglądarek, między innymi Google Chrome, Firefox, Safari, a nawet Internet Explorer

Wady:

- Brak oficjalnego wsparcia dla TypeScript
- Nieintuicyjne czekanie na oczekiwaną wartość

Selenium

```
// Click the button
const button = await driver.findElement(By.id("myButton"));
await button.click();

// Check updated textbox value
const textbox = await driver.findElement(By.id("myTextbox"));
const val = await textbox.getAttribute("value");
assert.strictEqual(value, "Updated Value");
```

✗ Taki kod może w niektórych przypadkach nie zadziałać

Selenium

```
// Click the button
const button = await driver.findElement(By.id("myButton"));
await button.click();

// Check updated textbox value
const textbox = await driver.findElement(By.id("myTextbox"));

// Wait until checkbox becomes the wanted value
await driver.wait(async () => {
    const val = await textbox.getAttribute("value");
    return val === "Updated Value";
}, 5000);
const val = await textbox.getAttribute("value");
assert.strictEqual(value, "Updated Value");
```

Cypress

Zalety:

- Oficjalne wsparcie dla TypeScript
- Zaimplementowany domyślny, automatyczny sposób czekania na oczekiwaną wartość
- Udostępnia wiele narzędzi do inspekcji przebiegu testów

Wady:

- Duża część narzędzi jest dostępna tylko w dodatkowo płatnym Cypress Cloud
- W pełni oficjalnie wspierana jest tylko przeglądarka Google Chrome, z ograniczonym wsparciem dla przeglądarek Firefox i Safari

Cypress

```
// Click the button  
cy.get('#myButton').click();
```

```
// Check updated textbox value  
cy.get<HTMLInputElement>('#myTextbox').should('have.value', 'Updated Value');
```

✓ Taki kod zaczeka na oczekiwaną wartość przez ustalony z góry czas (np. domyślnie 30 s), nie trzeba samemu pisać kodu czekającego, jak to miało miejsce w Selenium

Puppeteer

Zalety:

- Oficjalne wsparcie dla TypeScript

Wady:

- Aktualnie kompatybilny tylko z Google Chrome i Firefox
- Podobnie jak w Selenium, nieintuicyjne czekanie na zmianę wartości

Puppeteer

```
// Click the button
await page.click("#myButton");

// Wait until textbox value becomes the expected value
await page.waitForFunction(
  () => (document.getElementById("myTextbox")
    as HTMLInputElement)?.value === "Updated Value",
  { timeout: 5000 }
);

// Get updated textbox value
const value = await page.$eval("#myTextbox",
  el => (el as HTMLInputElement).value);
assert.strictEqual(value, "Updated Value");
```

Playwright

Zalety:

- Kompatybilność ze wszystkimi współcześnie popularnymi przeglądarkami (Google Chrome, Firefox i Safari), jak i również z ich wersjami mobilnymi
- Oficjalne wsparcie dla TypeScript
- Domyślne czekanie na zmianę wartości na oczekiwaną
- Łatwa inspekcja przebiegu testów

Wady:

- Trochę gorsze narzędzia do inspekcji niż Cypress w płatnym wariancie

Playwright

```
// Click the button
```

```
await page.click('#myButton');
```

```
// Check updated textbox value
```

```
const textbox = page.locator('#myTextbox');
```

```
await expect(textbox).toHaveValue('Updated Value');
```

✓ Podobnie jak w Cypressie, tutaj również program automatycznie zaczeka przez ustalony czas

Podsumowanie

Po analizie różnych dostępnych rozwiązań, zdecydowaliśmy się na zastosowanie narzędzia **Playwright**.



WYBRANE SZCZEGÓŁY IMPLEMENTACYJNE W APLIKACJI

Aplikacja wykorzystuje najnowsze dostępne narzędzia w ekosystemie React.

- **React 19 RC:** Wykorzystanie funkcji biblioteki przed oficjalnym stabilnym wydaniem
- **Vite:** wydajny build tool
- **Ekosystem TanStack:**
 - @tanstack/react-query: Zarządzanie stanem serwera, caching
 - @tanstack/react-table: Headless UI do budowania skomplikowanych tabel
 - @tanstack/react-form: Zarządzanie stanem formularzy i walidacją
 - @tanstack/react-router: Type-safe routing
- **Tailwind CSS:** Najnowsza wersja silnika CSS

Połączenie TanStack Table z React Form.

```
// frontend/.../FormAResearchAreaSection.tsx
{
  header: 'Rejon prowadzenia badań',
  cell: ({ row }) => (
    <form.Field
      name={`researchAreaDescriptions[${row.index}].differentName`}
      children={({field}) => (
        <AppInput
          value={field.state.value}
          onChange={field.handleChange}
          errors={field.state.meta.errors} // Walidacja per wiersz
        />
      )}
    />
  )
}
```

Warunkowe renderowanie przycisków akcji w oparciu o status.

```
// frontend/.../CruiseDetailsPage.tsx
switch (cruiseQuery.data?.status) {
  case 'Nowy':
    return (
      <>
        <AppButton onClick={() => setEditMode(true)}>Edytuj</AppButton>
        <AppButton onClick={() => setIsConfirmModal(true)}>Zatwierdź</AppButton>
      </>
    );
  case 'Potwierdzony':
    return (
      <AppButton onClick={() => setIsRevertModal(true)}>Cofnij status</
AppButton>
    );
}
```

Enkapsulacja logiki sprawdzania połączenia z API w prosty hook.

```
// modules/core/hooks/NetworkStatusApiHook.tsx
export function useNetworkStatusMutation({ setNetworkConnectionStatus }:
Props) {
  // Wywołanie prostego endpointu 'health'
  return useMutation({
    mutationFn: async () => client.get('/health'),
    onSuccess: () => setNetworkConnectionStatus(true),
    onError: () => setNetworkConnectionStatus(false),
    retry: false,
  });
}
```

Silne typowanie danych przesyłanych między formularzem a API.

```
// modules/cruise-schedule/models/CruiseFormDto.ts
export type CruiseFormDto = {
  startDate: string;
  endDate: string;
  managersTeam: {
    mainCruiseManagerId: string;
    mainDeputyManagerId: string;
  };
  cruiseApplicationsIds: string[];
  status?: string;
  shipUnavailable: boolean;
};
```


Transformacja danych domenowych na wydarzenia w kalendarzu.

```
// modules/cruise-schedule/components/CruiseCalendar.tsx
const events = React.useMemo(
  () =>
    cruises.map((cruise) => ({
      title: getTitle(cruise),
      start: new Date(cruise.startDate),
      end: new Date(cruise.endDate),
      link: `/cruises/${cruise.id}`,
      // Dynamiczne kolorowanie w zależności od typu
      color: cruise.shipUnavailable ? `#a10000` :
getHashColor(cruise.title),
    })),
  [cruises]
);
return <AppCalendar events={events} />;
```

Deklaratywne sprawdzanie uprawnień na poziomie routingu.

```
// frontend/src/routes/usermanagement.tsx
export const Route = createFileRoute('/usermanagement')({
  component: UserManagementPage,
  // Blokada przed załadowaniem JS dla strony
  beforeLoad: allowOnly.withRoles(Role.Administrator, Role.ShipOwner),
});

// frontend/src/modules/core/lib/guards.ts
allowOnly: {
  withRoles: (...roles: Role[]) => async ({ context }) => {
    // Weryfikacja ról w kontekście użytkownika
    if (!roles.some((r) => context.userContext!.currentUser!.roles.includes(r))) {
      throw redirect({ to: '/' });
    }
  }
}
```

Strategia dynamicznego wyboru komponentu tabeli w zależności od szerokości ekranu.

```
// frontend/src/modules/core/components/table/AppTable.tsx
export function AppTable<T>(props: Props<T>) {
  const { width } = useWindowSize(); // Custom hook

  // Wybór implementacji: klasyczna tabela webowa vs lista kart na mobile
  const TableComponent = width < 768 ? AppMobileTable : AppDesktopTable;

  return (
    <TableComponent
      table={table} // Wspólna instancja TanStack Table
      variant={props.variant}
      // ...
    />
  );
}
```

Automatyczne odświeżanie tokena (Silent Refresh) przy błędzie 401.

```
// frontend/src/modules/user/providers/UserContextProvider.tsx
React.useEffect(() => {
  const interceptorId = createAuthRefreshInterceptor(
    client,
    async (failedRequest) => {
      // 1. Próba odświeżenia tokena
      await context.refreshUser();

      // 2. Ponowienie oryginalnego zapytania z nowym tokenem
      failedRequest.response.config.headers['Authorization'] =
        `Bearer ${getStoredAuthDetails()?.accessToken}`;
      return failedRequest;
    },
    { statusCodes: [401], pauseInstanceWhileRefreshing: true }
  );
  return () => client.interceptors.response.eject(interceptorId);
}, [context]);
```

Zaawansowane testy z wykorzystaniem Playwright i wzorca Page Object Model.

```
// frontend/tests/fixtures/fixtures.ts
export const formTest = test.extend<{ formAPage: FormAPage }>({
  formAPage: async ({ page }, use) => {
    // Wstrzykiwanie gotowego obiektu strony (POM)
    const formAPage = await FormAPage.create(page);
    await use(formAPage);
  },
});

// frontend/tests/formA.spec.ts
test('valid form A', async ({ formAPage }) => {
  // Testy czytelne jak język naturalny
  await formAPage.fillForm();
  await formAPage.submitForm({ expectedResult: 'valid' });
});
```

Obsługa krytycznych błędów (Error Boundaries).

```
// frontend/src/modules/core/components/layout/AppErrorHandler.tsx
export function AppErrorHandler({ error }: ErrorComponentProps) {
  return (
    <AppLayout title={'Wystąpił nieoczekiwany błąd'}>
      { /* ... */ }
      <div className="font-semibold">{error.message}</div>
      <div className="text-center">
        Prosimy o maila na adres <AppLink>rejsy.help@ug.edu.pl</AppLink>.
      </div>
      <AppButton href="/">Przejdź do strony głównej</AppButton>
    </AppLayout>
  );
}
```

Detale i subtelne animacje.

```
// frontend/src/modules/core/components/layout/AppNavbar.tsx
<motion.header
  initial={{ y: -100 }} // Animacja wjazdu nagłówka
  animate={{ y: 0 }}
  transition={{ duration: 0.3 }}
>
  {/* ... */}
  <motion.div whileHover={{ scale: 1.1 }}> {/* subtelna animacja */}
    <AppLink href="/">Portal R/V Oceanograf</AppLink>
  </motion.div>

  <AnimatePresence>
    {userContext.currentUser && (
      <motion.div exit={{ scale: 0 }}> {/* Płynne znikanie */}
        <AppButton onClick={signOut}>Wyloguj</AppButton>
      </motion.div>
    )}
  </AnimatePresence>
</motion.header>
```

Dziękujemy za uwagę!